Informe Laboratorio 1

Sección 1

Benjamín Zúñiga Pueller
e-mail: benjamin.zuniga2@mail.udp.cl

Agosto de 2024

Índice

1. Descripción **2**

2. Actividades **2**
2.1. Algoritmo de cifrado 2
2.2. Modo stealth 2
2.3. MitM 3

3. Desarrollo de Actividades **4**
3.1. Actividad 1 4
3.2. Actividad 2 4
3.3. Actividad 3 6

1. Descripción

1. Usted empieza a trabajar en una empresa tecnológica que se jacta de poseer sistemas que permiten identificar filtraciones de información a través de Deep Packet Inspection (DPI). A usted le han encomendado auditar si efectivamente estos sistemas son capaces de detectar las filtraciones a través de tráfico de red. Debido a que el programa ping es ampliamente utilizado desde dentro y hacia fuera de la empresa, su tarea será crear un software que permita replicar tráfico generado por el programa ping con su configuración por defecto, pero con fragmentos de información confidencial. Recuerde que al comparar tráfico real con el generado no debe gatillar alarmas. De todas formas, deberá hacer una prueba de concepto, en la cual se demuestre que al conocer el algoritmo, será fácil determinar el mensaje en claro. Para los pasos 1,2,3 indicar el texto entregado a IA Generativa y validar si el código resultante cumple con lo requerido.

2. Actividades

2.1. Algoritmo de cifrado

1. Generar un programa, en python3 utilizando IA Generativa, que permita cifrar texto utilizando el algoritmo Cesar. Como parámetros de su programa deberá ingresar el string a cifrar y luego el desplazamiento.

```
~/Desktop sudo python3 cesar.py "criptografia y seguridad en redes" 9
larycxpajorj h bnpdarmjm nw anmnb
```

2.2. Modo stealth

1. Generar un programa, en python3 utilizando IA Generativa, que permita enviar los caracteres del string (el del paso 1) en varios paquetes ICMP request (un caracter por paquete en el campo data de ICMP) para de esta forma no gatillar sospechas sobre la filtración de datos. Deberá mostrar los campos de un ping real previo y posterior al suyo y demostrar que su tráfico consideró todos los aspectos para pasar desapercibido.

```
~/Desktop sudo python3 pingv4.py "larycxpajorj h bnpdarmjm nw anmnb"
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
```

El último carácter del mensaje se transmite como una b.

```
▾ Data (48 bytes)
    Data: 626009000000000010111213141516171819 1a1b1c1d1e1f20212223242526272
    [Length: 48]

0000  ff ff ff ff ff ff 00 00  00 00 00 00 08 00 45 00   ........ ......E·
0010  00 54 00 01 00 00 40 01  76 9b 7f 00 00 01 7f 06   ·T····@· v·......
0020  06 06 08 00 56 83 00 01  00 21 64 22 13 05 00 00   ····V··· ·!d"····
0030  00 00 62 60 09 00 00 00  00 00 10 11 12 13 14 15   ··b`···· ........
0040  16 17 18 19 1a 1b 1c 1d  1e 1f 20 21 22 23 24 25   ........ ·· !"#$%
0050  26 27 28 29 2a 2b 2c 2d  2e 2f 30 31 32 33 34 35   &'()*+,- ./012345
0060  36 37                                              67
```

2.3. MitM

1. Generar un programa, en python3 utilizando IA Generativa, que permita obtener el mensaje transmitido en el paso2. Como no se sabe cual es el desplazamiento utilizado, genere todas las combinaciones posibles e imprímalas, indicando en verde la opción más probable de ser el mensaje en claro.

```
Ξ ~/Desktop Ξ sudo python3 readv2.py cesar.pcapng
0        larycxpajorj h bnpdarmjm nw anmnb
1        kzqxbwozinqi g amoczqlil mv zmlma
2        jypwavnyhmph f zlnbypkhk lu ylklz
3        ixovzumxglog e ykmaxojgj kt xkjky
4        hwnuytlwfknf d xjlzwnifi js wjijx
5        gvmtxskvejme c wikyvmheh ir vihiw
6        fulswrjudild b vhjxulgdg hq uhghv
7        etkrvqitchkc a ugiwtkfcf gp tgfgu
8        dsjquphsbgjb z tfhvsjebe fo sfeft
9        criptografia y seguridad en redes
10       bqhosnfqzehz x rdftqhczc dm qdcdr
11       apgnrmepydgy w qcespgbyb cl pcbcq
12       zofmqldoxcfx v pbdrofaxa bk obabp
13       ynelpkcnwbew u oacqnezwz aj nazao
14       xmdkojbmvadv t nzbpmdyvy zi mzyzn
15       wlcjnialuzcu s myaolcxux yh lyxym
16       vkbimhzktybt r lxznkbwtw xg kxwxl
17       ujahlgyjsxas q kwymjavsv wf jwvwk
18       tizgkfxirwzr p jvxlizuru ve ivuvj
19       shyfjewhqvyq o iuwkhytqt ud hutui
20       rgxeidvgpuxp n htvjgxsps tc gtsth
21       qfwdhcufotwo m gsuifwror sb fsrsg
22       pevcgbtensvn l frthevqnq ra erqrf
23       odubfasdmrum k eqsgdupmp qz dqpqe
24       nctaezrclqtl j dprfctolo py cpopd
25       mbszdyqbkpsk i coqebsnkn ox bonoc
```

Finalmente, deberá indicar por que eligió esa IA Generativa en relación a otras del mercado. Comente si tuvo algún problema al momento de generar los códigos.

3. Desarrollo de Actividades

Para el desarrollo de estas actividades se utilizo el LLM Gemini en su versión Pro.

3.1. Actividad 1

Para dar inicio a esta primera fase, se utilizó la inteligencia artificial Gemini para generar un programa en Python 3 que implementara el algoritmo de cifrado César. A este *script* se le ingresan dos parámetros: una cadena de texto (*string*) correspondiente a la palabra a cifrar, y un valor numérico entero que define el desplazamiento. El requerimiento se formuló mediante el siguiente *prompt*:

> "Necesito un programa en python 3 que cifre un texto utilizando el algoritmo de cifrado Cesar. Este programa se le ingresara un parámetro String que sera la palabra a cifrar y ademas el desplazamiento que se va a usar"

Posteriormente, se procedió a ejecutar el programa generado ingresando el texto «Hola Larva» junto con el desplazamiento seleccionado. Como se puede apreciar en la Figura 1, el programa cifró el texto exitosamente, entregando como resultado la cadena «Qxuj ujaej», la cual quedó lista para ser utilizada en la siguiente etapa.

```
benjaminzunigapueller@Benjamins-MacBook-Air Laboratorio1 % python3 cesar.py "Hola larva" 9
Qxuj ujaej
```

Figura 1: Ejecución y resultados del programa Cesar.py

A nivel de arquitectura, el *script* (`cesar.py`) implementa el algoritmo iterando sobre cada carácter ingresado. Aplica la transposición matemática correspondiente únicamente a las letras del alfabeto (respetando el uso de mayúsculas y minúsculas). Los espacios y signos de puntuación se conservan intactos, garantizando la integridad del mensaje para la fase de inyección.

3.2. Actividad 2

Con la frase cifrada obtenida en la etapa anterior, la segunda actividad consistió en desarrollar un método de exfiltración sigilosa. Se solicitó a Gemini la creación de un programa capaz de realizar la inyección de estos datos. El *script* fue configurado para enviar el tráfico hacia la IP 8.8.8.8, correspondiente al servidor DNS de Google. Esto se llevó a cabo mediante el siguiente *prompt*:

> ”Funciono perfecto, ahora en mi segunda actividad me piden esto: Generar un programa, en python3 utilizando IA Generativa, que permita enviar los caracteres del string (el del paso 1) en varios paquetes ICMP request (un carácter por paquete en el campo data de ICMP) para de esta forma no gatillar sospechas sobre la filtración de datos.”

Para lograr esta evasión, el *script* (`pingv4.py`) hace uso de la librería *scapy* para forjar paquetes *Echo Request*. Para cumplir con los requerimientos de sigilo y mantener la coherencia forense, el código estructura el *payload* de 56 bytes conservando los primeros 8 bytes intactos (reservados para el *timestamp*) y el bloque final desde el byte `0x10` hasta `0x37`. La inyección del carácter exfiltrado se realiza estratégicamente en el offset número 8, logrando evadir sospechas sobre la manipulación de datos.

Con el programa implementado, se dio inicio a la exfiltración de datos enviando cada carácter de la frase de forma individual a través de paquetes ICMP. La captura de red de este procedimiento se puede observar en la Figura 2.

No.	Time	Source	Destination	Protocol	Length	Info
134	12.770694	172.20.10.6	8.8.8.8	ICMP	98	Echo (ping) request id=0x3039, seq=1/256, ttl=64 (reply in 152)
157	14.598094	172.20.10.6	8.8.8.8	ICMP	98	Echo (ping) request id=0x3039, seq=2/512, ttl=64 (reply in 158)
159	15.649078	172.20.10.6	8.8.8.8	ICMP	98	Echo (ping) request id=0x3039, seq=3/768, ttl=64 (reply in 160)
162	16.714568	172.20.10.6	8.8.8.8	ICMP	98	Echo (ping) request id=0x3039, seq=4/1024, ttl=64 (reply in 163)
165	17.764430	172.20.10.6	8.8.8.8	ICMP	98	Echo (ping) request id=0x3039, seq=5/1280, ttl=64 (reply in 166)
179	18.814411	172.20.10.6	8.8.8.8	ICMP	98	Echo (ping) request id=0x3039, seq=6/1536, ttl=64 (reply in 180)
200	19.866327	172.20.10.6	8.8.8.8	ICMP	98	Echo (ping) request id=0x3039, seq=7/1792, ttl=64 (reply in 201)
216	20.915409	172.20.10.6	8.8.8.8	ICMP	98	Echo (ping) request id=0x3039, seq=8/2048, ttl=64 (reply in 217)
218	21.962355	172.20.10.6	8.8.8.8	ICMP	98	Echo (ping) request id=0x3039, seq=9/2304, ttl=64 (reply in 219)
223	23.014401	172.20.10.6	8.8.8.8	ICMP	98	Echo (ping) request id=0x3039, seq=10/2560, ttl=64 (reply in 224)

Figura 2: Visualización de inyección de tráfico en Wireshark

El análisis inicial (Figura 2) revela que el flujo de los 10 paquetes ICMP —cifra equivalente a la longitud del *string* transmitido— se mantiene constante entre la IP origen 172.20.10.6 y la IP destino 8.8.8.8. Adicionalmente, la Figura 3 ilustra una inspección más profunda del campo de datos: los recuadros oscuros confirman la modificación del primer byte en los tres paquetes iniciales, demostrando la inyección secuencial de los caracteres de la frase.

Figura 3: Visualización detallada del payload de los paquetes ICMP en Wireshark

3.3. Actividad 3

Tras haber inyectado y capturado exitosamente el tráfico de red, el objetivo de la tercera etapa fue realizar el proceso inverso de criptoanálisis. Se solicitó nuevamente a Gemini la creación de un programa que leyera el archivo de captura (`.pcapng`) almacenado en Wireshark, extrajera el mensaje oculto transmitido en la actividad dos, y lo descifrara. Para esto se utilizó el siguiente *prompt*:

> .Ahora tenemos que hacer esta parte: Generar un programa, en python3 utilizando IA Generativa, que permita obtener el mensaje transmitido en el paso 2. Como no se sabe cual es el desplazamiento utilizado, genere todas las combinaciones posibles e imprímalas, indicando en verde la opción más probable de ser el mensaje en claro."

El *script* de criptoanálisis (`readv2.py`) opera leyendo el archivo `.pcapng` y aislando los paquetes ICMP pertinentes. Extrae el byte modificado de cada *payload* para reconstruir la cadena original. Al desconocer la llave (desplazamiento), ejecuta un ataque de fuerza bruta. El sistema integra un motor de evaluación heurística que puntúa la presencia de vocabulario en español, logrando identificar de manera autónoma el mensaje en claro (texto plano) y la llave utilizada, destacándolos en la salida estándar.

Como resultado final, la Figura 4 ilustra la salida por consola del programa automatizado, detallando las 26 permutaciones posibles aplicando fuerza bruta sobre el cifrado César. Entre ellas, gracias a la evaluación heurística del *script*, la cadena de texto descifrada con éxito se identificó correctamente y se encuentra resaltada en color azul para su fácil visualización.

```
● (venv) benjaminzunigapueller@Benjamins-MacBook-Air Laboratorio1 % sudo python3 readv2.py Captura.pcapng
  0  Qxuj ujaej
  1  Pwti tizdi
  2  Ovsh shych
  3  Nurg rgxbg
  4  Mtqf qfwaf
  5  Lspe pevze
  6  Krod oduyd
  7  Jqnc nctxc
  8  Ipmb mbswb
  9  Hola larva
  10 Gnkz kzquz
  11 Fmjy jypty
  12 Elix ixosx
  13 Dkhw hwnrw
  14 Cjgv gvmqv
  15 Bifu fulpu
  16 Ahet etkot
  17 Zgds dsjns
  18 Yfcr crimr
  19 Xebq bqhlq
  20 Wdap apgkp
  21 Vczo zofjo
  22 Ubyn ynein
  23 Taxm xmdhm
  24 Szwl wlcgl
  25 Ryvk vkbfk
```

Figura 4: Descifrado por fuerza bruta ejecutado por el programa readv2.py

Conclusiones y comentarios

Durante el desarrollo del presente laboratorio, la implementación teórica de canales encubiertos presentó las siguientes complicaciones técnicas y analíticas que requirieron soluciones específicas:

- **Gestión de privilegios y entornos virtuales:** La forja de paquetes ICMP con la librería *scapy* exige la creación de *raw sockets*, requiriendo permisos de superusuario (`sudo`). Esto generó conflictos con las dependencias del usuario, resolviéndose mediante la creación de un entorno virtual aislado y forzando la ejecución del binario de Python con privilegios elevados.

- **Evasión de detección y simulación de tráfico:** Para que la exfiltración pasara desapercibida, no era factible enviar paquetes de un solo byte. El desafío se superó implementando un algoritmo que inyecta 55 bytes de relleno (*padding*), imitando el tamaño exacto (56 bytes de *payload*) y la cadencia de un ping nativo del sistema operativo.

- **Automatización del criptoanálisis:** En la etapa de extracción, al desconocer la clave del cifrado César, el descifrado no podía ser manual. Se desarrolló un motor de

evaluación heurística que somete a prueba las 26 combinaciones posibles, analizando la frecuencia de letras y detectando palabras clave para identificar automáticamente el mensaje en claro.

- **Anomalías forenses a nivel de *payload*:** Aunque se logró evadir la detección por tamaño, el análisis forense profundo reveló una complicación inherente: al inyectar el carácter en el primer byte de datos, se sobrescribe el espacio utilizado por el sistema operativo para el *timestamp* (marca de tiempo). Esta discrepancia fue la huella clave para evidenciar la manipulación del tráfico en Wireshark.